

Google Calendar Setup — Peak Studios CO

This document walks the **owner** through setting up the Google Calendar OAuth credentials our booking system needs. At the end you'll send three values to your developer:

1. `GOOGLE_CALENDAR_CLIENT_ID`
2. `GOOGLE_CALENDAR_CLIENT_SECRET`
3. `GOOGLE_CALENDAR_REFRESH_TOKEN`

You'll do this **once**. After that the booking system stays connected forever — no re-auth needed.

Sign in to all of the following with `peaklegacyvideos@gmail.com`, since that's the calendar we're connecting to.

Step 1 — Create a Google Cloud project

1. Go to <https://console.cloud.google.com/>.
2. In the top bar, click the project dropdown → **New Project**.
3. Name it something like "Peak Studios Bookings". Leave organization blank if asked.
4. Click **Create**, then make sure the new project is selected in the top-bar dropdown.

Step 2 — Enable the Google Calendar API

1. In the left sidebar, go to **APIs & Services** → **Library**.
2. Search for "Google Calendar API".
3. Click it, then click **Enable**.

Step 3 — Configure the OAuth consent screen

1. Sidebar → **APIs & Services** → **OAuth consent screen**.
2. Click **Get started**.
3. Fill in:
 - **App name**: Peak Studios CO Bookings
 - **User support email**: peaklegacyvideos@gmail.com
 - **Audience**: External
 - **Contact information**: peaklegacyvideos@gmail.com
 - Agree to the user data policy.

4. Click **Create**.
5. After saving, find the **Audience** tab in the sidebar and under **Test users**, click **+ Add users** and add `peaklegacyvideos@gmail.com`. (Required while the app is in "Testing" mode — which is fine for our use, since only you ever sign in.)

Step 4 — Create OAuth 2.0 Client credentials

1. Sidebar → **APIs & Services** → **Credentials**.
2. Click **+ Create credentials** → **OAuth client ID**.
3. **Application type**: Web application.
4. **Name**: "Bookings Refresh Token Helper" (anything is fine).
5. Under **Authorized redirect URIs**, click **+ Add URI** and paste exactly:

```
https://developers.google.com/oauthplayground
```

6. Click **Create**.
7. Copy the **Client ID** and **Client secret** that appear in the dialog. Keep this tab open or paste them into a temporary note — you'll use both in the next step.

Step 5 — Get a refresh token via OAuth Playground

1. Open <https://developers.google.com/oauthplayground/> in a new tab.
2. Top-right, click the gear icon (⚙️ **OAuth 2.0 configuration**).
3. Tick **Use your own OAuth credentials**.
4. Paste the **Client ID** and **Client Secret** from Step 4 into the matching fields.
5. Close the gear panel.
6. In the left list ("Step 1 Select & authorize APIs"), scroll down to **Calendar API v3** and tick `https://www.googleapis.com/auth/calendar`.
7. Click **Authorize APIs**. Sign in with `peaklegacyvideos@gmail.com` and click **Allow**.
 - If you see a "Google hasn't verified this app" warning, click **Advanced** → **Go to Peak Studios CO Bookings (unsafe)**. This is expected because the app is in Testing mode.
8. You'll be redirected back to the playground at "Step 2 Exchange authorization code for tokens".
9. Click **Exchange authorization code for tokens**.
10. Copy the **Refresh token** that appears (it's the value next to `"refresh_token":` — a long string starting with `1//`).

Step 6 — Send three values to your developer

Send these three values to Bryce **securely** (1Password, signed Slack DM, encrypted email — not plain email):

Key	Where you got it
GOOGLE_CALENDAR_CLIENT_ID	Step 4
GOOGLE_CALENDAR_CLIENT_SECRET	Step 4
GOOGLE_CALENDAR_REFRESH_TOKEN	Step 5

Once Bryce drops them into the booking system, your live calendar becomes the source of truth for available times — anything you block out (existing meetings, personal events, vacation) automatically disappears from the booking page.

Dev steps once credentials arrive

When the owner sends the values, the developer:

1. Open `.env.local` and set:

```
GOOGLE_CALENDAR_CLIENT_ID=...
GOOGLE_CALENDAR_CLIENT_SECRET=...
GOOGLE_CALENDAR_REFRESH_TOKEN=1//...
GOOGLE_CALENDAR_ID=primary
```

(`GOOGLE_CALENDAR_ID` defaults to `primary` , which means the main calendar of `peaklegacyvideos@gmail.com` . Override only if the owner wants a separate sub-calendar dedicated to bookings.)

2. Restart the dev server (`pnpm dev`). The Google Calendar client in `lib/google-calendar.ts` is initialized lazily on first use, so no other config touch is needed.
3. **Verify availability reads** by hitting the endpoint directly:

```
curl "http://localhost:3000/api/availability?date=2026-04-28"
```

- Without creds: returns all 16 slots (fallback path).
 - With creds: should still return all 16 if that day is empty in the calendar. Block out 10:00–11:00 in the owner's GCal for that date and re-fetch — the 10:00 AM and 10:30 AM slots should disappear.
4. **Verify event creation** by completing a test booking in the modal. Within a few seconds, an event titled `Discovery call – {first} {last}` should appear in `peaklegacyvideos@gmail.com` 's calendar at the booked time. The lead's email is added as an attendee, and the description includes phone, package interest, and notes.

5. For staging/production, set the same four vars in Vercel project settings → Environment Variables.
-

FAQ

Do I have to "publish" the app in Google Cloud?

No. Leaving it in "Testing" mode is fine because only you (`peaklegacyvideos@gmail.com`) ever sign in. Test-mode refresh tokens last as long as you don't revoke access on your Google account.

What if I rotate or revoke the token?

Just redo Steps 4–6 and send the new three values. The system can be updated without any redeploy by swapping environment variables.

Can someone read my calendar with these tokens?

Anyone with all three values could read/write your `peaklegacyvideos@gmail.com` calendar. Treat them like a password.

What permissions did I grant?

The full Calendar scope (`auth/calendar`) — read availability and create/update events on the connected account's calendar. The booking system only ever:

- Reads busy times to show free slots to leads
- Creates one event per booked discovery call